

System e-commerce sklepu odzieżowego

Dynamika, Aktorzy oraz Architektura Wdrożeniowa i Komponentowa

Kacper Gumulak & Damian Judka

2.1 Identyfikacja głównych aktorów systemu



Customer (Klient)

Aktor zewnętrzny. Przegląda asortyment, zarządza koszykiem, adresami i finalizuje zamówienia.



User (Pracownik / Admin)

Aktor wewnętrzny. Zarządza produktami, wysyłkami, zwrotami i monitoruje Event Sourcing.



Payment Provider

Aktor systemowy zewnętrzny. Obsługuje transakcje finansowe i weryfikuje ich status.



Shipping Provider

Firma Kurierska. Odbiera zlecenia wysyłki i dostarcza numery listów (tracking URL).

2.2 Główne Przypadki Użycia (Use Cases)



UC1: Zarządzanie koszykiem Modyfikacja zawartości z weryfikacją ilości (`check_sellable_product_quantity`).



UC2: Złożenie zamówienia (Checkout) Uzupełnienie danych logistycznych i wywołanie procedury `add_order` (status `placed`).



UC3: Realizacja płatności Po sukcesie u Providera, `pay_order` generuje fakturę (invoices) i zmienia status na `paid`.



UC4: Obsługa logistyczna Pracownik wywołuje `prepare_order_shipment` z trackingiem, a potem `ship_order`.



UC5: Obsługa zwrotów i anulacji `cancel_order` lub `return_order` wyzwalają odpowiednie wpisy w `stock_events`.

Diagram Przypadków Użycia

Wizualizacja Zależności

Interakcje z systemem e-commerce podzielone na role:

- **Customer** inicjuje zakupy i dokonuje płatności.
- **User / Admin** zajmuje się logistyką i ofertą.
- Wywołania bazodanowe (triggery i procedury) automatyzują komunikację z podmiotami zewnętrznymi (**Payment & Shipping Provider**).

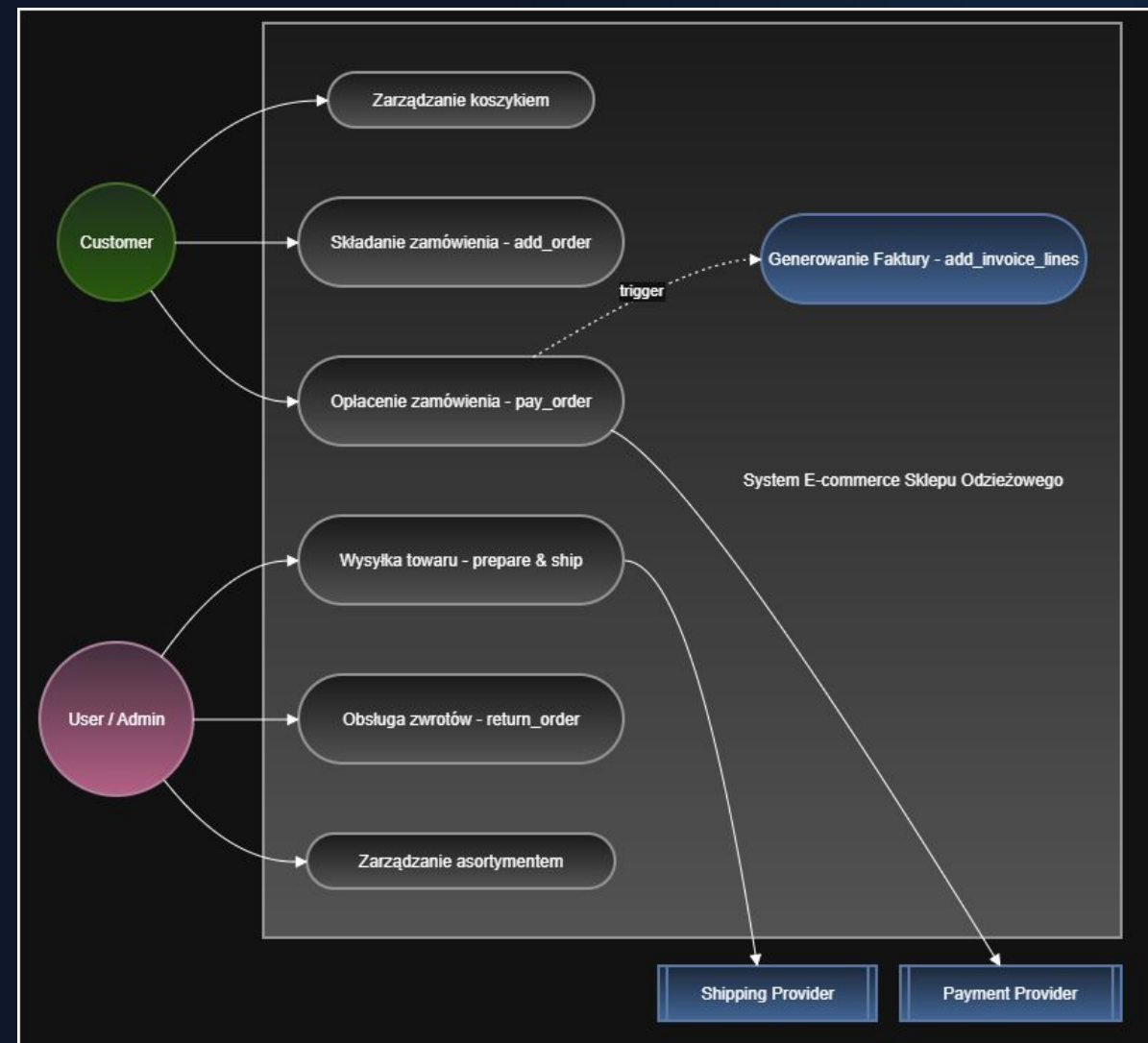


Diagram Czynności (Activity)

Przepływ procesu płatności

Krok po kroku automatyzacja finalizacji zamówienia:

- Weryfikacja w bramce (decyzja: Odrzucona vs Zatwierdzona).
- Ścieżka sukcesu odpala logikę z **CALL pay_order()**.
- Automatyczne generowanie **Invoices** w tle.

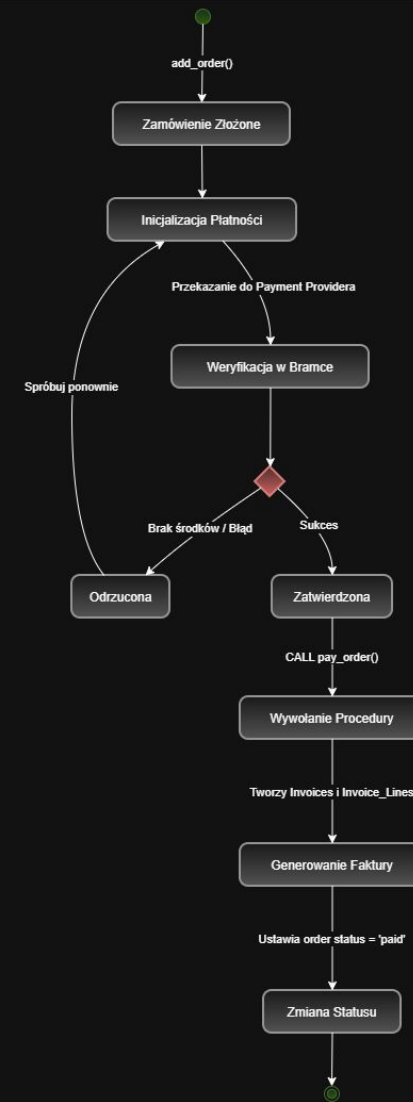
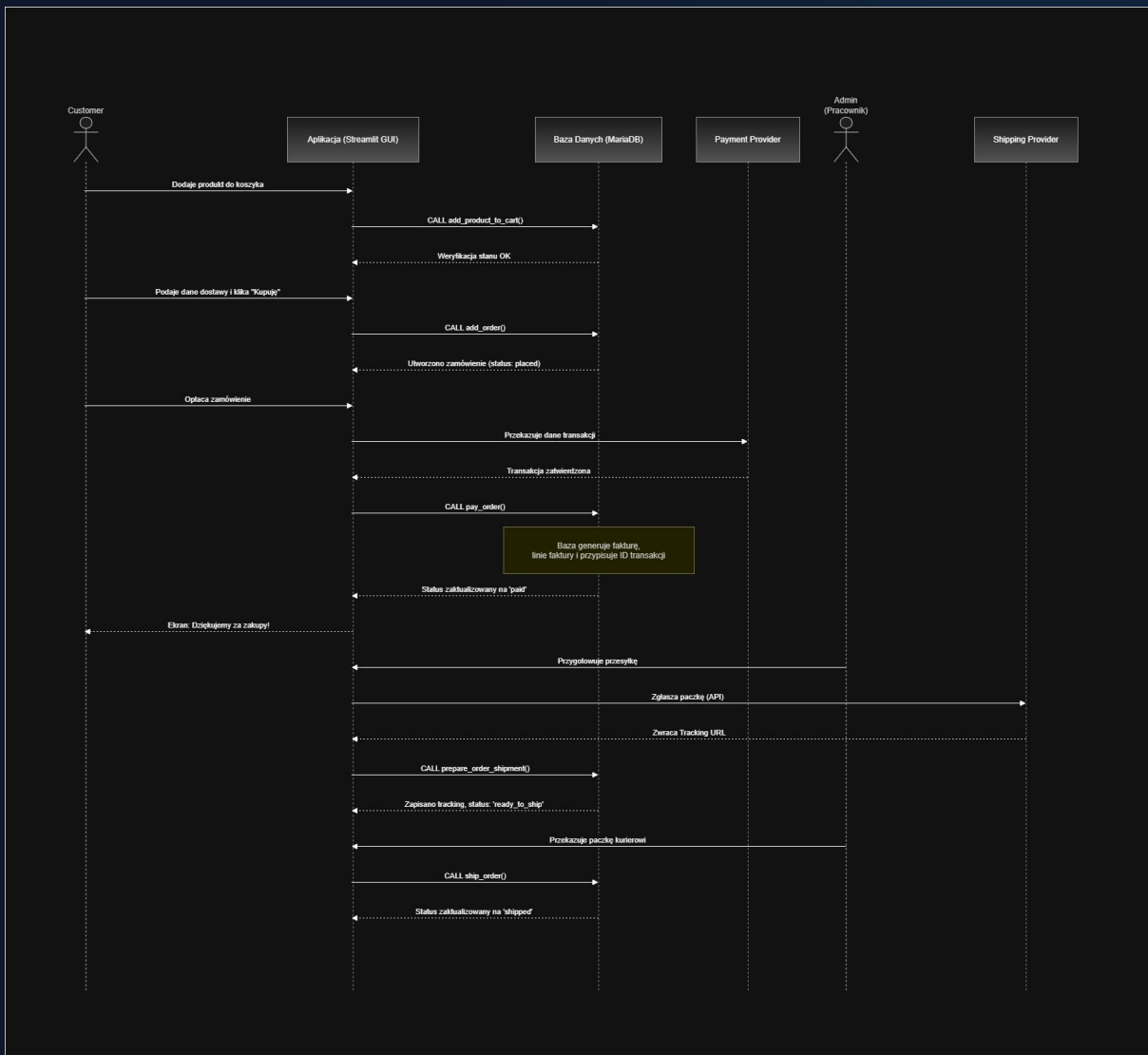


Diagram Interakcji (Sequence)

Sekwencja wywołań API i DB

Przepływ czasu i żądań w architekturze:

- **Aplikacja (GUI)** zleca operacje bezpośrednio w bazie MariaDB (Stored Procedures).
- Baza samodzielnie pilnuje reguł integralności stanu (ACID).
- Bramka płatnicza asynchronicznie przesyła potwierdzenia, co zwalnia blokady na froncie.



3.1 Architektura Wdrożenia (Deployment)



Urządzenie Końcowe

Przeglądarka Web. Łączy się z systemem z zewnątrz wyłącznie poprzez standardowy protokół HTTP/HTTPS.



Kontener Aplikacji (Frontend)

Środowisko oparte na Python Streamlit. Pełni rolę bezstanowego pośrednika i lekkiego klienta.



Kontener Bazy Danych

Serce systemu: MariaDB 11. Wykonuje zaawansowane procedury składowane i operacje transakcyjne (InnoDB).



Izolacja i Persystencja

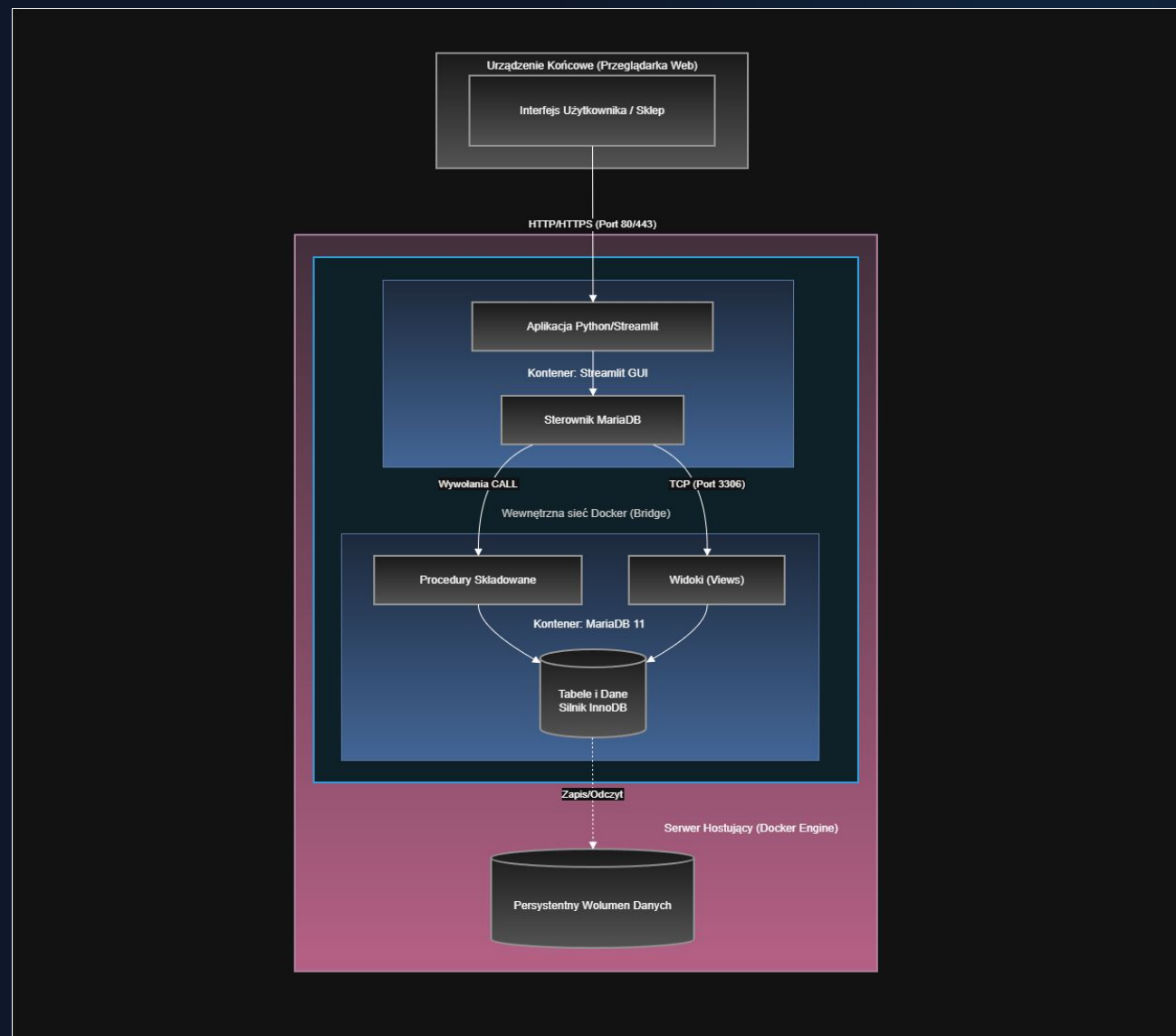
Zamknięta sieć Docker Network oraz Wolumeny Danych gwarantujące bezpieczeństwo i odporność na restarty.

Diagram Wdrożenia (Deployment Diagram)

Fizyczna infrastruktura węzłów

System wykorzystuje konteneryzację do ujednolicenia środowisk (dev/prod):

- Architektura ukrywa serwer bazy danych głęboko w prywatnej sieci kontenerowej.
- Jedynek punktem wejścia z zewnątrz jest **Aplikacja Streamlit**.
- Trwałość logów Event Sourcing i zamówień gwarantują podpięte z hosta **Wolumeny Danych**.



3.2 Ostateczna struktura komponentów

Odstępstwo od klasyki

Zamiast klasycznego trójwarstwowego (Frontend, API, DB), zintegrowaliśmy logikę bezpośrednio z warstwą danych w bazie MariaDB.

Mamy **Moduł Interfejsu (Streamlit)**, który jedynie renderuje widoki (np. `v_product_details`) i zbiera formularze.

Moduły Główne

DB Connector: Łącznik autoryzujący w Python.

Stored Procedures: Silnik logiki biznesowej, tu dzieje się cała "magia" (`add_order`, `pay_order`).

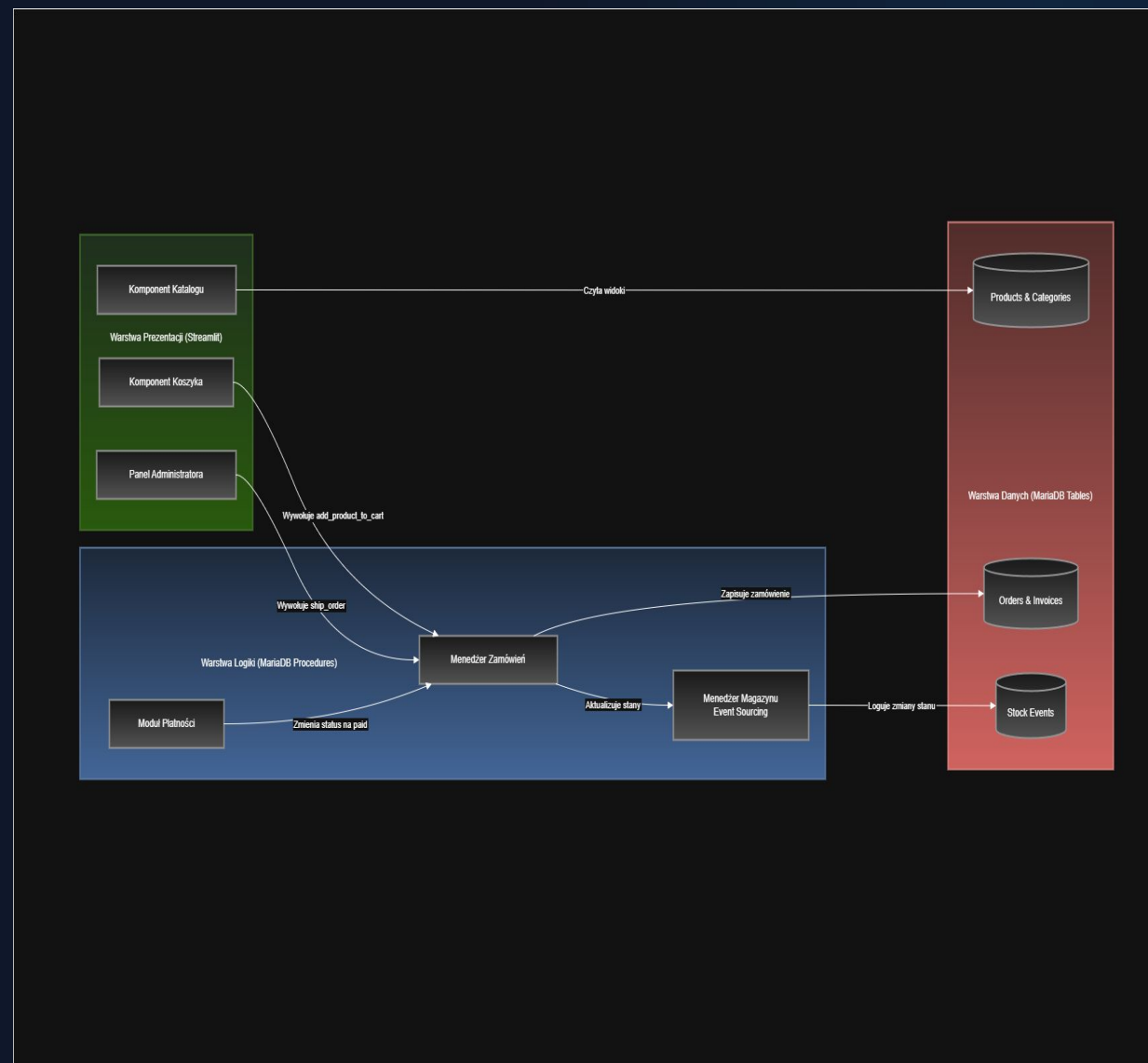
Data Storage: Struktury relacyjne wymuszające integralność referencyjną.

Diagram Komponentów

Zależności pakietów w kodzie

Model komponentów ilustruje przepływ danych wewnątrz zmodyfikowanej architektury:

- **Interfejs GUI** wywołuje bezpośrednio przygotowane przez inżynierów procedury składowane.
- Warstwa bazy danych jest całkowicie autonomiczna i pilnuje bezpieczeństwa operacji.
- Wysoka modularność pozwala na ewentualną zmianę frontendu w przyszłości bez dotykania logiki biznesowej.



Dziękujemy za uwagę

To kompletny przegląd architektury i dynamiki naszego systemu.

Zapraszamy do zadawania pytań.